Name : AKASH SAMPATH

Topic :

Can you find out a way to change the port number the application runs on WITHOUT directly changing application.properties? Dynamically only using env vars?

Note :

- This is actually how apps are meant to be configured in docker and production environment
- So we need not touch application.properties at all . Where the spring boot overrides everything using environment variable

CORE IDEA :

- Our app has two layers of configuration : Default and Runtime :
- In default it is based on code inside and runtime is based on outside the app
- So in default which is inside our code , by using application.properties we use : server.port=8080
- In runtime which is outside our app , we use : SERVER_PORT=9090 and this is live override in runtime and if we fall back or go back we use default

BASED ON MAPPING OUR APP :

- So the concept is : Using default , we use server.port=8080 and using override we use SERVER_PORT=9090
- So finally the app runs on 9090

SPRING BOOT UNDERSTANDS BY :

- Spring boot automatically converts : server,port to SERVER_PORT
- So when it sees SERVER_PORT=9090 , it overrides in application.properties as server.port=8080
- So when we start a application by using application.properties , we use server.port=8080 and while using environment variables , we use SERVER_PORT=9090
- So the application compares both and based on high priority , the environment variable wins and thus the app runs on 9090
- So when no environment variable Is set , No SERVER_PORT is provided , so the app uses application.properties and thus runs on 8080
- So environment is the real control and application.properties is the backup plan

ENVIRONMENT VARIABLE :

- The line that can change your spring boot app port by not changing code and not editng application.properties
- So simply we use this SERVER_PORT=9090

IDEA :

- Spring boot reads config from many places , but gives highest priority to environment variable
- So if two values exist like application.properties which uses server.port=8080 and environment variables uses SERVER_PORT=9090
- So finally spring boot chooses : 9090 in environment variable

SPRING BOOT AUTOMATICALLY MAPS :

- Based on conversion rules in application.properties and environment variable
- So application.properties converts to environment variable and vice versa

In application.properties	In environment variable
server.port	SERVER_PORT
spring.datasource.url	SPRING_DATASOURCE_URL

BASED ON APPLICATIONS :

- As application.properties uses default instructions to run port 8080
- But in environment variable it overrides the default instructions to run on port 9090

WHERE TO PLACE :

In docker compose : In environment :

```
environment :
      SERVER_PORT : 9090
```

WHY AND HOW THE WORKING :

IDEA :

- Spring boot is flexible in how configurations name are written , so same property , different forms :

server.port	SERVER_PORT	server_port	SERVER.PORT

- So all these are same thing to spring boot , So internally spring converts everything into one standard format

BASED ON CONVERSION RULE :

- Spring boot automatically converts server.port to SERVER_PORT
- So dot (.) become (_) , and lowercase becomes UPPERCASE

PROCEDURE HAPPENING INTERNALLY :

So when set :

```
environment :
      SERVER_PORT : 9090
```

- Spring boot reads all environment variables and converts SERVER_PORT to server.port
- So it check configuration based on priority such as environment variables port 9090 and for application.properties with port 8080
- So finally server,port = 9090 , thus app runs on 9090

EXAMPLE :

- In our application.properties we use server.port=8080
- So in docker environment variable we use SERVER_PORT : 7070
- So finally app runs on 7070 and not on 8080

THE FLOW :

- Using environment variables as SERVER_PORT=9090 , the spring boot converts to server.port=9090 which overrides application.properties so thus makes app runs on 9090

DOCKER COMPOSE :

- It is like a control panel where you start container and connect them and pass config like environment variable using docker compose like how I run my app

ENVIRONMENT VARIABLE FIT IN : DOCKER COMPOSE :

In docker compose :

```
environment:
      SERVER_PORT : 9090
```

- This means it tells container when you start use SERVER_PORT=9090

SPRING BOOT UNDERSTANDING :

- Spring boot sees SERVER_PORT and converts to server.port , so internally your app behaves as if using application.properties for server.port=9090

FULL FLOW :

When you run docker compose up , Docker reads your file with :

```
environment :
      SERVER_PORT : 9090
```

- The container starts with that variable inside the container with SERVER_PORT =9090
- So spring boot reads it and says , I have found SERVER_PORT , I will override server.port
- So finally app runs on the port 9090

IMPORTANT :

```
ports:
    - "8080:8080"

environment:
      SERVER_PORT :9090
```

- Our laptop is 9090 : Container is 9090

BASED ON MATCHING :

Using :

```
ports:
   - "8080:8080"

  environment:
     SERVER_PORT:9090
```

- So the app runs on 9090 but docker exposes 8080, which the browser looks for localhost:8080 but found nothing

By using :

```
app:
 build:
 ports:
  - "9090:9090"

 environment:
    SERVER_PORT:9090
```

- So this will work , as docker compose uses environment variable by spring boot which reads the environment variable and overrides config which makes the app runs
- So docker compose sets the environment and spring boot follows it

WHAT HAPPENS NEXT :

So we configured :

```
ports:
 - "9090:9090"

 environment:
    SERVER_PORT:9090
```

- So when docker starts your container , it passes this inside SERVER_PORT : 9090

So the spring boot reads and starts and think :

- Do I have server.port in application.properties and if yes , maybe 8080
- Do I have SERVER_PORT in environment variable and if yes use 9090

Because of environment variable 9090 is chosen

- Now inside the container , the app is running on port 9090
- The line “9090:9090” means our laptop using 9090 : Also container using 9090
- So finally we open browser by typing : http://localhost:9090

REQUEST FLOW :

- So our browser uses localhost:9090 and using our system which is docker host maps the ports with 9090:9090 to spring boot container and the app listens on 9090 and response comes back

FLOW :

- The browser uses localhost:9090 using docker host based on 9090:9090 to the container where app runs on 9090

CASES OF REMEMBER :

CASE 1 : Changed environment variable but not ports :

```
ports:
 - "8080:8080"

 environment:
    SERVER_PORT:9090
```

- The app runs on 9090 , docker exposes 8080 and so browser uses localhost:8080 and container uses 8080 , but the app is not there , so connection failed

CASE 2 : Changed ports but not environment variable :

```
ports:
 - "9090:9090"

 environment:
    SERVER_PORT:8080
```

- So the app still runs on default : 8080 , so browser uses 9090 and container uses 9090 but the app runs on 8080 , but connection fails

IMPORTANT :

- **So if we config like this :**

```
ports:
 - "8080:8080"

 environment:
   SERVER_PORT:9090
```

- We can expect the app should run and open http://localhost:8080 , but actually happening is your app runs on 9090 because of SERVER_PORT and docker exposes 8080 , so these are not same

STEP BY STEP BREAKDOWN :

- Using docker mapping for host with 8080 and container with 8080
- So by using spring boot the app runs on 9090
- So therefore , the request opens to 8080 , but app is on 9090 so it will fail

WRONG SETUP :

- As browser uses localhost : 8080 with docker forwards to container : 8080 which is inside container making app actually on 9090

CORRECT SETUP :

- **Must match everything like :**

```
ports:
 - "9090:9090"

 environment:
   SERVER_PORT : 9090
```

- So the browser uses localhost : 9090 using docker forwards go container with 9090 inside container with the app running on 9090 , So connection is successful

USING DIFFERENT HOST AND CONTAINER PORTS :

```
ports:
 - "8001:9090"

 environment:
   SERVER_PORT:9090
```

- The app runs inside the container with port 9090 and you can access from browser with port 8081 and this works perfectly

MATCHING :

- So , HOST_PORT : CONTAINER_PORT = SERVER_PORT (Inside the app)

SERVER_PORT :

- This is inside spring boot and where your app actually listens : like :

```
environment:
   SERVER_PORT:9090
```

- So the app is running on port 9090 inside the container

CONTAINER_PORT :

- Based on docker side , where the port and docker expect inside the container : like **:**

```
      ports:
       - "9090:9090"
```

- So the right side 9090 is the container port

HOST_PORT :

- Our laptop access is done by using : http://localhost:9090 , where left side 9090 is the host port
- So If docker does the mapping on : HOST_PORT and CONTAINER_PORT which is to be matched and important because app is listening on the container port

WRONG :

```
ports:
 - "8080:8080"

 environment:
   SERVER_PORT : 9090
```

- So in this the browser uses localhost : 8080 using docker forwards to container : 8080 , but app is not running on 9090

CORRECT :

```
ports:
 - "9090:9090"

 environment:
    SERVER_PORT: 9090
```

- In this browser uses localhost : 9090 using docker with container : 9090 by spring boot , the app listens on 9090

FLEXIBLE :

```
ports:
 - "8085:9090"

 environment:
  SERVER_PORT:9090
```

- In this the app inside using 9090 and container port using 9090 , And host port is using port : 8085
- So finally we can access using http://localhost:8085

MULTIPLE PORTS :

- Helps by not changing one app port again and again
- Just running multiple containers of the same app , with each different port
- So we use same code with different settings to run multiple running apps

CONCEPT :

- Like Running 2 containers of your spring boot app
- Where container 1 runs on port 8081 and container 2 runs on port 8082

IN DOCKER COMPOSE :

We use first service as app1 , with container_name : app1 and using : ports :

```
ports:
 - "8081:8081”
```

With environment :

```
environment:
   SERVER_PORT: 8081
```

We use second service as app2 , with container_name : app2 and using ports :

```
ports:
 - "8082:8082”
```

With environment :

```
environment:
   SERVER_PORT:8082
```

HAPPENING INTERNALLY :

- In app1 runs inside container using 8081 and can access with http://localhost:8081
- In app2 runs inside container using 8082 and can access with http://localhost:8082

IMPORTANT :

- Each container must have unique HOST PORT (ports) and unique SERVER_PORT , otherwise there will be conflict

When using same ports for both app1 and app2 will cause port already in use with port :

```
ports:
 - "8080:8080"
```

So using ports :

```
ports:
 - "8001:8080"

 environment:
   SERVER_PORT : 8080
```

So Inside container using port 8080 and outside using browser using port 8081 and also using ports :

```
ports:
 - "8082:8080"

 environment:
   SERVER_PORT: 8080
```

- Inside container using port 8080 and outside using browser using port 8082

FLOW :

- Using browser we are using localhost :8081 and localhost:8082 , so use both or anyone
- So localhost:8081 used by app1 and localhost:8082 used by app2
- So both app1 and app2 are connected to the same mysql:3306

```
spring.datasource.url=jdbc:mysql://mysql:3306/electricity_db
```

OVERRIDE ANY CONFIGURATION : IN RUNTIME USING ENVIRONMENT VARIABLES : BY SPRING BOOT :

- Spring boot maps with application.properties with environment variables :

So based on CONVERSION RULES :

In application.properties	In environment variable
(.) dot	_ (underscore)
lowercase	UPPERCASE

EXAMPLE :

DATABASE URL :

application.properties	environment variable
spring.datasource.url=jdbc:mysql://mysql:3306/electricity_db	SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/electricity_db

USERNAME :

application.properties	environment variable
spring.datasource.username=root	SPRING_DATASOURCE_USERNAME: root

SERVER PORT :

application.properties	environment variable
server.port=8080	SERVER_PORT=8080

PRIORITY ORDER :

Spring boot check config in the order :

1. CLI
2. Environment Variable
3. application.properties

- So we are using Environment Variables , So this is given more priority over application.properties

REAL LIFE :

- So without Environment Variable : If we want change in DB , We must edit the whole code

So with Environment Variable : We just change :

```
SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/electricity_db
```

So the same app now connects to production DB :

```
SPRING_DATASOURCE_URL: jdbc:mysql://prod-db:3306/prod
```

DIFFERENT ENVIRONMENTS :

Environments	**Database**
DEV	localhost
TEST	test-db
PROD	prod-db

- So therefore , Only the environment changes , but the code in database stays same

CODES :

DOCKER COMPOSE :

```
version: '3.8'

services:

  mysql:
    image: mysql:8.0
    container_name: mysql-electricityakash
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: electricity_db
    ports:
      - "3307:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  app:
    build: .
    container_name: electricity-appakash
    restart: always
    depends_on:
      - mysql
    ports:
      - "9090:9090"
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/electricity_db
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: root
      SERVER_PORT: 9090

volumes:
  mysql_data:
```

EXPLANATION : Docker Compose

Section	Commands	Explanation
Compose File	version: '3.8'	Defines docker compose file format version used widely and very stable
Services	services:	Define containers like app , each service is one container , Have 2 services such as mysql for DB and app for spring boot app
MYSQL	mysql:	First service name , can use as hostname
MYSQL	image: mysql:8.0	Pulls official image of MYSQL from DockerHub
MYSQL	container_name:	Custom name for container
MYSQL	restart: always	If container stops or crashes , restart automatically
MYSQL	environment:	Passes variables into MYSQL container by setting password and creating DB
MYSQL	ports: - "3307:3306"	Port Mapping with host : container , 3307 (Laptop) and 3306 (MYSQL inside container) From laptop : localhost:3307 From app container : mysql:3306
MYSQL	volumes:	Stores MYSQL data without this data is lost and with this data is saved
APP	app:	Name of your application service
APP	build: .	Build docker image from project folder using docker file
APP	container_name:	Custom name for app container
APP	depends_on:	Start MYSQL before app
APP	ports: - "9090:9090"	9090 (Browser) 9090 (App inside container)
APP	environment:	DB Connection by connecting to service mysql with port : 3306 with credentials

APP	SERVER_PORT: 9090	Overrides server.port =8080 in application.properties
APP	volumes:	Declares volume used earlier
WHY 9090?	SERVER_PORT: 9090	App runs on port 9090 instead of 8080

IMPORTANT CONCEPTS :

- We can use service name as hostname also
- HOST : CONTAINER for port mapping
- SERVER_PORT overrides server.port
- Use service name as mysql only and not localhost

MULTIPLE PORTS :

```
app1:
  ports:
    - "8081:8081"
  environment:
    SERVER_PORT: 8081

app2:
  ports:
    - "8082:8082"
  environment:
    SERVER_PORT: 8082
```

EXPLANATION : MUTLPLE PORTS :

Note :

- We are creating 2 app containers such as app1 runs on port 8081 and app2 runs on port 8082
- So both are same apps , but running separately

app1:	Service name , docker creates a container named like app1 , Inside docker network , other services can use app1
ports: - "8081:8081"	This is port mapping with format as HOST_PORT : CONTAINER_PORT In left side (8081) for our laptop or browser In right side (8081) inside container Open browser with http ://localhost:8081 Request goes to container port 8081
environment: SERVER_PORT: 8081	Telling spring boot to run app on port 8081 so normally server.port=8080 is default so this is overrided by SERVER_PORT : 8081 So inside container , app runs on port 8081
app2:	Another sperate container
ports: - "8082:8082"	By mapping : Browser : localhost:8082 and container : port 8082
environment: SERVER_PORT: 8082	Inside container app runs on 8082

CONCEPTS :

Service	**Port (Internal)**	**Port (External)**	**URL**
app1	8081	8081	localhost:8081
app2	8082	8082	localhost:8082

- Each container is isolated like app1 and app2 are separate , and they don't conflict even if same code is given to both
- We change using SERVER_PORT for app1 with 8081 and app2 with port 8082 to avoid conflict and make both app run on 8081 and 8082
- Ports mapping must match like "8081:8081" because app must run on 8081 inside the container so , SERVER_PORT = 8081 is required